

Compiled Proof Guidance for Repository-Scale Verified Software Synthesis

A Deterministic-Authority Study on Vero

Mohit Mahajan (ByteVerity / Synthesa)

September 2026

Abstract

Repository-scale verified software synthesis is usually approached as an end-to-end agent task: a model edits implementations and proofs, observes compiler feedback, and tries again. We study a different architecture in which the model, when used, has no proof authority. Synthesa resolves Vero specifications through Lean’s elaborated environment, compiles them into semantic obligation graphs, classifies parent-contract adequacy, applies a bounded deterministic proof floor, and promotes only kernel-checked helpers into a content-addressed library. Exact residuals can request a narrowly typed proposal; the response is pinned before Lean either admits or refutes it. Candidate slot changes then pass through a signed, terminal-once build plan and Vero’s independent clean re-render and grader. In an iterative, benchmark-seen campaign at Vero commit 0a7325d and Lean 4.29.1, the resulting mixed-mode portfolio improves from 2/43 complete repositories and 1030/2705 accepted specifications to 40/43 and 2606/2705, with no Synthesa-only result counted as a Vero pass. The selected artifacts comprise 37 proof-mode and six codeproof-mode evaluations, so 40/43 is not presented as a uniform-mode leaderboard result. Thirty-three metered external proposer calls cost USD 0.7104 in total; frozen artifact replay makes zero model calls, while human, Codex-orchestration, and local-compute costs were not fully metered. Three residual repositories reveal an equally important result: fail-closed elaboration and parent-adequacy checks distinguish proof-search exhaustion from malformed specifications and missing semantic contracts. The evidence supports compiled repository organization as a promising complement to generative proof search, while motivating a future blind, uniform-mode evaluation.

Contents

1. Introduction	2
2. Problem formulation and trust model	3
2.1 Repository-level obligations	3
2.2 Authority	3
2.3 Boundedness	3
3. Compiled proof-guidance architecture	4
3.1 Elaboration-first obligation extraction	4
3.2 Parent contracts and GIGO protection	4
3.3 Deterministic proof floor and residual normalization	4
3.4 Semantic obligation graph	5
3.5 Oracle and certificate compilation	5
3.6 Verified lemma library	5
3.7 Bounded proposer ABI	5
3.8 Signed Act and independent grading	6
4. Experimental method	6

4.1 Environment	6
4.2 Starting and stopping points	6
4.3 Modes	6
4.4 Metrics	6
4.5 Cost accounting	7
5. Results	7
5.1 Aggregate outcome	7
5.2 Ntheory: one abstraction chain, five residual projections	7
5.3 Reedsolo: bounded computation plus algebraic seams	7
5.4 Shared helpers as repository infrastructure	7
5.5 Honest residuals	8
6. Analysis	8
6.1 Representation precedes proof search	8
6.2 Compile organization, generate only abstractions	8
6.3 Precompiled public documents are useful but not magical	8
6.4 GIGO is a product capability	9
6.5 What zero-call replay buys	9
7. Relationship to prior work	9
8. Business and product implications	9
9. Limitations and threats to validity	10
10. Next experiment	10
11. Conclusion	10
Data and artifact availability	11

1. Introduction

Machine-checked verification changes the authority structure of software generation. A model can suggest an implementation or proof, but validity is a property of a formal artifact checked by a small trusted kernel. This makes verified synthesis a natural setting for separating creative proposal from acceptance. The separation is especially important at repository scale, where proof obligations share definitions, invariants, and helper chains across modules.

Vero makes this repository-level problem concrete. It contains 43 Lean 4 projects, 743 APIs, and 2,705 formal specifications translated from real software repositories. It supports proof-only evaluation against supplied reference implementations and code-and-proof evaluation in which implementations are also synthesized (Ye, Lou, et al. 2026). Its strongest reported configuration completes 27 repositories in codeproof and 25 in proof mode, despite accepting more than 85% of individual specifications. Vero’s analysis attributes this gap to repository organization: completed repositories place a median of more than 70% of proof lines in helper lemmas, and almost every successful repository shares helpers across specifications (Ye, Lou, et al. 2026).

That observation suggests a systems question. Must a model discover the entire repository proof architecture by free-form trial and error, or can much of the organization be compiled? Synthesa explores the latter. It treats repository verification as a collection of bounded decisions connected by checked seams: which declaration a specification denotes, whether its type elaborates, which implementation symbols and parent laws it depends on, which goals share a semantic shape, which verified abstraction covers them, and whether an exact candidate may be admitted. The potentially unbounded creative step is narrowed to proposing a missing abstraction only after deterministic decomposition.

We make four contributions:

1. We describe an elaboration-first representation of repository proof obligations and a structured residual taxonomy that avoids source-text guessing.
2. We present a compiled proof-guidance architecture combining deterministic tactics, parent-contract adequacy, semantic clustering, precompiled formal oracles, verified helper reuse, and an optional bounded proposer.
3. We define an authority chain in which proposal bytes are pinned, Lean alone admits proofs, signed plans constrain materialization, and Vero’s untouched grader remains the benchmark verdict.
4. We report a fully disclosed, iterative Vero campaign that produces a 40/43 mixed-mode qualified portfolio and 2606/2705 accepted specifications, together with its costs, artifacts, and limitations.

This is not a claim of a blind 40/43 single-mode agent run. It is a systems study of whether repository-scale organization can be moved from an agent into a deterministic proof-guided product.

2. Problem formulation and trust model

2.1 Repository-level obligations

Let a Vero repository provide an implementation bundle I , specifications $S = \{s_1, \dots, s_n\}$, imports M , and an editable slot schedule E . In proof mode, the target is the fixed canonical implementation I_c and each accepted slot establishes either $s_i(I_c)$ or its permitted audit polarity. In codeproof mode, the candidate chooses the editable implementation I and must prove the specifications or submit one of Vero’s exact audit forms.

A repository passes only when every required specification passes the independent grader. This all-or-nothing repository metric means local proof coverage and global completion are materially different objectives. A system that independently solves easy s_i may score well by specification while leaving most repositories incomplete.

2.2 Authority

Our governing rule is:

```
model = untrusted proposer
deterministic/formal validator = authority
```

Every candidate follows:

```
pinned -> built -> checked -> admitted or refuted
```

The model cannot declare a theorem proven, change the allowed edit surface, waive a residual, introduce a new axiom, or define the benchmark result. Similarly, a Synthesa receipt proves identity, authorization, and lineage; it does not independently reproduce Lean semantics. Vero’s own clean render, build, axiom screening, and report remain the external score authority.

2.3 Boundedness

Benchmarks must expose a finite set of repositories, declarations, slots, and grading outcomes. That does not make every theorem decidable, but it creates many bounded control problems around theorem construction. Synthesa compiles those control decisions: obligation identity, dependency coverage, candidate selection, tactic budgets, adapter completeness, certificate coverage, slot authorization, and result admission. Mathematical cores are handled by Lean, SMT/SyGuS-backed synthesis where appropriate, finite reflected certificates, or a narrowly scoped proposal. The product never equates a bounded workflow with a proof of an unbounded mathematical proposition.

3. Compiled proof-guidance architecture

3.1 Elaboration-first obligation extraction

Early experiments showed that source syntax is an unsafe proof-planning interface. Names can be qualified differently after elaboration; implicit arguments, coercions, typeclass instances, and reducible definitions can change the actual goal. The system therefore queries Lean’s environment for every `spec_*` declaration and serializes:

- fully qualified declaration name and universe parameters;
- all explicit and implicit binders with elaborated types;
- the target expression;
- referenced declarations and implementation heads;
- imports, source location, and dependency neighborhood;
- render, toolchain, and elaboration-environment digests.

An obligation is eligible for proof construction only if the declaration resolves, its type elaborates, its implementation binding is identified, and the record remains stable across clean renders. No source-text fallback is allowed. This converts a generic build failure into a precise distinction between a malformed input, an unknown declaration, and a genuine proof goal.

3.2 Parent contracts and GIGO protection

A reusable theorem needs an explicit parent world: types, operations, laws, and representation bridges sufficient for its statement and proof. We compile finite adequacy surfaces whose cells answer whether each required clause is bound in the frozen source, needs a bridge lemma, is inapplicable, or is missing. Unknown cells default to `GIGO_HOLD`.

This gate prevents a dangerous failure mode. When an intended law is absent, repeated tactics or a powerful model may produce increasingly plausible text, but no axiom-clean proof follows. The system instead emits a minimal residual such as `MISSING_EDGE_ORIENTATION_BRIDGE`, `MISSING_PROGRESS_MEASURE`, or `BENCHMARK_SPEC_DOES_NOT_ELABORATE`. A missing assumption is not silently inserted into the theorem.

GIGO is distinct from Vero’s formal audit. A formal audit requires an accepted Lean proof that a fixed reference violates a spec, a spec is unsatisfiable, or a set is inconsistent. Under-specification or opacity may establish neither polarity and therefore remains unfilled.

3.3 Deterministic proof floor and residual normalization

Before any model call, a frozen portfolio of Lean tactics and typed proof constructors is tried under bounded time and resource limits. The portfolio is intentionally stable across a qualification stage so a change in coverage can be attributed to representation or knowledge rather than tactic shopping.

Rejected candidates are normalized into structured residuals, including:

<code>UNRESOLVED_GOAL</code>	<code>MISSING_REWRITE</code>
<code>MISSING_CASE_SPLIT</code>	<code>INDUCTION_REQUIRED</code>
<code>TYPE_MISMATCH</code>	<code>MISSING_INSTANCE</code>
<code>ARITHMETIC_RESIDUAL</code>	<code>UNKNOWN_DECLARATION</code>
<code>TERMINATION_ISSUE</code>	<code>DEPENDENCY_CYCLE</code>

Normalization retains the exact Lean diagnostic and a digest. The class is a routing signal, not a replacement for the diagnostic. It supports deterministic selection of the next constructor, library query, adequacy check, or proposal type.

3.4 Semantic obligation graph

The repository is compiled into a graph with nodes for specifications, implementation APIs, elaborated declarations, residuals, candidate abstractions, verified lemmas, and module/import units. Edges record reference, implementation binding, import, proof dependency, candidate coverage, and verified discharge.

Clustering operates on semantic features rather than filenames alone:

- target head and normalized proposition shape;
- binder type heads and shared state types;
- referenced definitions and recursive predicates;
- implementation head;
- dependency neighborhood; and
- failed tactic/residual signature.

The output is a bounded assertion such as “these nine unresolved obligations share state X , transition T , and preservation-shaped targets,” not a claim that any particular invariant is true. That distinction leaves abstraction invention open while compiling the repository-organization problem.

3.5 Oracle and certificate compilation

We use “oracle” to mean a pinned, checkable knowledge artifact, not an authority that can override Lean. Candidate sources include:

- existing Lean libraries;
- public formal source documents pinned by URL, revision, license, and digest;
- finite truth tables or certificates for bounded domains;
- CEGIS/SMT/SyGuS-produced candidates with checkable witnesses;
- previously verified repository helpers; and
- one narrowly typed model proposal.

Every source-derived theorem needs a repository adapter. A public Coq theorem, for example, can explain intended FLoCq semantics but does not prove a proposition about a differently translated opaque Lean constant. The adapter must establish the representation and operation equations in the frozen environment. Adapter adequacy is a separate, fail-closed seam.

Finite certificates can be atomized. Instead of trusting a generated table as a monolith, the system binds the domain, one certificate row per input, the decision rule, coverage proof, and projection theorem. This was useful for finite-field reasoning: computation supplies bounded evidence while Lean checks how it implies each specification.

3.6 Verified lemma library

An accepted helper is stored with exact source bytes, proposition, imports, dependency roots, Lean/toolchain/environment digest, axiom report, covered specifications, goal-shape tags, and verification receipt. The library admits only Lean-checked artifacts. Dependencies form an explicit acyclic graph and are topologically materialized.

The important unit of progress is therefore not a transient model response but a verified abstraction that can discharge several specifications—or several repositories—without further generation. The system rechecks the library before proposing new work.

3.7 Bounded proposer ABI

The proposer is invoked only after deterministic paths end at one exact semantic residual. A request contains:

- proposal class, such as helper lemma, invariant, proof skeleton, or proof-friendly implementation;
- exact elaborated binders and target;
- permitted imports and output grammar;

- verified local context and counterexamples;
- accepted and rejected examples;
- the smallest residual to repair; and
- repository, Vero, Lean, prompt, model, and configuration digests.

The response bytes are pinned before validation. A rejected response may produce a narrower residual but never mutates the verified library. If a proposal is admitted, the resulting theorem—not the model response—is retained as deterministic knowledge. Identical future residuals should reuse it with zero new calls.

3.8 Signed Act and independent grading

Candidate materialization remains in Synthesa’s existing actuator path:

```
software.plan -> signed plan -> software.build -> terminal receipt
```

The plan binds the benchmark, mode, source and sandbox digests, Vero and grader digests, Lean/Lake/Python environment, exact allowed slots, parent-world root, abstraction/library roots, builder binary, and terminal-once state. Build rechecks these values and refuses drift, tamper, unauthorized slots, and replay.

Finally, Vero extracts only scheduled slot bodies, re-renders from the frozen benchmark, drops unexpected markers, builds the clean project, and performs its axiom checks. Only that report is aggregated.

4. Experimental method

4.1 Environment

We froze Vero at commit `0a7325df9e9e6dbc275c0ad483b3d1cbe38d9b09` and used `leanprover/lean4:v4.29.1`. The final campaign compiler has SHA-256 `0653c7421f2aa681fb8d7795b400fc43cec2e3cfd24be85ecc928e921f2666aa`. The aggregate campaign root is `cf0bc418a41ca32eb5b8ede967d2e55c4be38ca514fc0b6e503369711ea3d8c`.

The study was iterative and benchmark-seen. Capabilities were developed in response to observed residual classes, then frozen and checked against further repositories where possible. This is product-oriented research, not a blind held-out agent evaluation. We preserve strict failures and distinguish generic compiler additions from repository proof artifacts, but the absence of a prospective holdout limits causal conclusions.

4.2 Starting and stopping points

The campaign starts from a frozen 43-repository portfolio with 2 complete repositories and 1030 accepted specifications. Subsequent results are promoted only when backed by an official report and, for the qualified Act path, a signed plan and receipt. We stopped at 40/43 and 2606/2705 rather than inventing parent laws for the final 99 obligations.

4.3 Modes

The selected final artifact is proof mode for 37 repositories and codeproof for six. This allows the systems study to use the strongest qualified artifact per repository, but it prevents direct comparison with one Vero mode. We therefore report three views: the mixed portfolio, the 37-repository proof subset, and the six-repository codeproof subset. A future leaderboard experiment must use a maintainer-approved uniform protocol.

4.4 Metrics

Primary measurements are complete repositories, accepted specifications, and false acceptance. Secondary measurements include proposal calls and cost, verified abstractions, residual classes, specs discharged per helper, and repository unlocks per generic abstraction. The last metric tests the central industrialization

hypothesis: if each new abstraction unlocks a repository, the method scales; if every tail repository needs bespoke theory, it does not.

4.5 Cost accounting

The evidence tree contains 33 uniquely priced NanoGPT proposer calls: 628,022 prompt tokens, 34,377 completion tokens, 662,399 total, and USD 0.710391762755. The models were Qwen 3.5 4B, DeepSeek V4 Flash, GPT-5.5, and Claude Sonnet 4.6. This conservative ledger includes rejected and diagnostic calls.

Frozen artifact grading uses no model. “Zero model calls” means zero calls in replay because accepted knowledge has been compiled into the artifact; it does not erase the 33 development calls. The dollar ledger also excludes Codex orchestration, human labor, and local compute and therefore is not total system cost.

5. Results

5.1 Aggregate outcome

Result view	Repositories	Full	Specifications	Spec rate
Starting portfolio	43	2	1030/2705	38.08%
Final mixed-mode portfolio	43	40	2606/2705	96.34%
Selected proof subset	37	35	2286/2382	95.97%
Selected codeproof subset	6	5	320/323	99.07%

The portfolio adds 38 complete repositories and 1,576 accepted specifications relative to its starting state. All 43 final artifacts were extracted using upstream **vero-extract**; corresponding report and artifact digests are provided for independent maintainer grading.

The aggregate is strong evidence that deterministic organization and reusable verified knowledge can materially improve a repository-scale proof campaign. It is not evidence that the same frozen system would score 40/43 on a new blind sample, nor that it beats 27/43 under Vero’s published single-mode budget.

5.2 Ntheory: one abstraction chain, five residual projections

Ntheory moved from 57/62 to 62/62 through a verified number-theory chain: prime factorization support, a character contract, Gauss/Eisenstein and quadratic reciprocity support, an abstract Jacobi factor product, and a refinement from the frozen recursive worker to that abstraction. The final foundation was projected into five residual specifications. The closing execution made no model call, though earlier R&D proposer calls are included in the global ledger. This case demonstrates amortization within a hard repository cluster.

5.3 Reedsolo: bounded computation plus algebraic seams

Reedsolo reached 48/48 in proof mode through a staged finite-field and polynomial-semantics program. A complete byte-domain certificate supplied finite evidence; Lean-checked field laws, polynomial evaluation, synthetic division, generator recurrence, root, and distance abstractions connected it to repository specifications. The key lesson is that finite enumeration alone was insufficient: its certificate needed atomized coverage and verified seams to the higher-level propositions.

5.4 Shared helpers as repository infrastructure

Across the campaign, difficult walls were approached by compiling shared parents before exact **prove_*** projections. Examples include ordered-map set-update laws, relation/path composition, codec roundtrips, periodic calendar oracles, order/canonicalization properties, and state-transition invariants. Successful parents became dependency-addressed inputs rather than prompt text. This qualitatively aligns with Vero’s observation that completed repositories are organized around shared helper libraries (Ye, Lou, et al. 2026).

5.5 Honest residuals

The final 99 specifications are concentrated in three repositories:

Repository	Mode	Accepted	Residual	Main finding
DedekindReals	proof	0/82	82	frozen spec module has a dependent type mismatch
FLoCq	proof	189/203	14	disconnected trusted wrappers and opaque inverse contracts
Verdict	codeproof	116/119	3	opaque non-editable conversion lacks preservation equations

These are not all equivalent. DedekindReals fails before obligations can be faithfully elaborated. FLoCq includes two other specifications already accepted through formal disproof, one translation-premise concern, twelve wrapper alignment gaps, and one native-Float inverse gap. Verdict’s exposed conversion type admits both satisfying and violating models, but neither polarity has been proved for the frozen opaque constant. Stopping preserves the distinction between “cannot prove,” “input is malformed,” and “formal audit accepted.”

6. Analysis

6.1 Representation precedes proof search

The first deterministic proof-floor attempt originally produced zero coverage because it did not operate on the true elaborated `spec_*` types. Expanding the tactic portfolio would have confounded the experiment. Resolving declarations, binders, targets, and implementation heads first converted build noise into meaningful proof residuals. This is a general lesson for proof agents: compiler feedback is only useful when attributed to a faithful semantic unit.

6.2 Compile organization, generate only abstractions

The model is poorly positioned to repeatedly rediscover repository structure. It is better used for the irreducible question “what lemma or invariant might connect these exact obligations?” after the system has compiled the cluster, known context, counterexamples, and output shape. This reduces authority and scope at once. The accepted result can then be reused without repeating the call.

6.3 Precompiled public documents are useful but not magical

Public formalizations, manuals, standards, and reference implementations can be compiled into candidate oracles. They accelerate theorem discovery and clarify intended semantics. They do not automatically cross a representation seam. FLoCq and Verdict show why: a public theorem about a valid Coq type or a Rust conversion cannot prove a theorem about an opaque or differently modeled Lean declaration until a checked adapter exists.

6.4 GIGO is a product capability

Failing closed on an under-specified parent may lower a nominal score, but it is essential for trustworthy automation. The same decision machinery used for software policies can compile the completeness surface of a proof adapter. Every declared requirement must have an explicit disposition; missing cells produce exact residuals. This prevents benchmark pressure from becoming a reason to manufacture assumptions.

6.5 What zero-call replay buys

Zero-call replay changes the economics and governance of verified artifacts. Once a lemma or finite certificate is admitted, future builds can use its content-addressed proof without provider availability, stochastic variation, or inference cost. An organization can audit which knowledge was introduced, which obligations it covers, and which compiler/kernel accepted it. This is amortization, not retroactive model erasure.

7. Relationship to prior work

Vero establishes repository-scale joint implementation and proof synthesis as a benchmark and shows that shared helper organization separates many full solves from partial ones (Ye, Lou, et al. 2026). We use Vero’s existing authority boundary and audit mechanism rather than replacing them, and focus on compiling the organization layer its evaluation identifies.

Verina studies joint generation of code, specifications, and proofs at a smaller task granularity and shows that iterative Lean feedback improves proof success but remains costly and limited even over many repair steps (Ye et al. 2025). Our residual normalizer has a related motivation, but narrows repair to elaborated repository obligations and retains accepted results as verified shared infrastructure.

VeriSpecGen decomposes natural-language intent into atomic requirements, associates tests with explicit traceability, and performs localized repair (Ye, Yang, et al. 2026). Synthesa applies an analogous decomposition principle on the other side of the specification boundary: formal declarations are elaborated, clustered, and linked to proof residuals and reusable abstractions. We do not synthesize Vero’s frozen specifications, and Lean proof validity is stronger than test-based intent evidence. The works are complementary.

CEGIS and SyGuS treat synthesis as bounded candidate generation refined by counterexamples and formal constraints (Solar-Lezama et al. 2006; Alur et al. 2013). Synthesa uses this style where domains and grammars are suitable, while allowing a model to propose abstractions outside the deterministic grammar. In both cases the candidate is subordinate to the verifier.

8. Business and product implications

The business value is not merely a higher benchmark number. The architecture turns one-time expert or model insight into governed reusable capital. A verified helper or compiled standards oracle can be reused across builds with no inference call, while lineage answers exactly which artifact, environment, and authority produced a result. This is attractive for regulated and high-assurance software where repeatability, reviewability, and refusal are as important as generation speed.

The workflow also separates expensive uncertainty from routine execution. Deterministic analysis handles obligation discovery, coverage, policy, dependency scheduling, and admission. Models are reserved for missing abstractions. Lean and domain validators settle truth. Signed Act constrains mutation. Independent graders or enterprise postconditions settle deployment success. This pattern extends beyond Lean to quality, security, compliance, and transformation domains when their decisions can be compiled into bounded oracle contracts with checkable seams.

Finally, the residual repositories illustrate commercial value in saying no. An automated system that invents a contract for an opaque conversion can appear productive while creating unreviewable risk. A precise hold—“the required preservation equation is absent from the frozen parent”—is an actionable engineering output.

It directs a spec owner to the smallest repair and avoids spending more model budget on an impossible request.

9. Limitations and threats to validity

First, this was an iterative engineering campaign with access to benchmark repositories and residual reports. Results may include benchmark-specific proof artifacts even though compiler logic is intended to remain generic. A blind holdout is needed to measure generalization.

Second, the final portfolio mixes modes. The proof and codeproof subsets cover different repositories, so their completion rates cannot be extrapolated to all 43. A leaderboard comparison requires a uniform mode or explicit organizer approval of portfolio selection.

Third, cost accounting is incomplete. The NanoGPT ledger is exact for the 33 captured calls, but orchestration, labor, and compute are not priced. Frozen replay cost is operationally useful but is not comparable to one-pass agent development cost.

Fourth, zero false acceptance is measured relative to the campaign’s official-report admission rule. It is not a proof that every specification captures intended source semantics. Indeed, the curator findings expose translation and parent-contract risks that only benchmark maintainers can resolve.

Fifth, source availability is split. The extracted artifacts and reports are prepared for private maintainer replication. The research compiler currently lives on a preserved dirty worktree rather than a clean release commit. A public research artifact should freeze and license a reproducible source snapshot or provide a minimal open agent wrapper.

Sixth, the campaign did not quantify total helper-line share, cross-repository reuse, or repository-unlocks-per-abstraction uniformly across all stages. Future runs should emit those metrics directly from signed artifact graphs.

10. Next experiment

The next scientifically clean experiment is not more work on the same 99 residual specifications. It is a frozen, uniform-mode, prospective campaign:

1. agree with Vero maintainers on proof or codeproof mode, time, compute, and cost accounting;
2. freeze the compiler source, binary, tactic portfolio, oracle library, proposer ABI, prompts, model versions, and public-document corpus;
3. select an unseen repository subset or newly curated Vero release;
4. run end to end with complete event, cost, residual, and lineage capture;
5. submit only **vero-extract** artifacts for independent grading; and
6. report ablations for deterministic floor, compiled-oracle reuse, small proposer, and stronger proposer.

The key causal metric should be new full repositories per newly admitted generic abstraction, alongside model calls per accepted specification. A positive cross-repository ratio with zero false acceptance would directly test whether compiled organization generalizes.

11. Conclusion

Vero asks whether agents can build formally verified repositories. Our study suggests a complementary question: how little authority and repository organization must be left to the agent? Synthesa compiles elaborated obligations, deterministic search, semantic clusters, parent adequacy, certificates, verified helper dependencies, and signed execution into a fail-closed workflow. Models propose only bounded missing abstractions; Lean decides proof validity; Vero decides the score.

The resulting 40/43, 2606/2705 mixed-mode portfolio is encouraging but must be read with its experimental qualifications. More important than the headline is the observed mechanism: verified abstractions can break multi-specification walls and become zero-call reusable infrastructure, while malformed or under-specified parents stop with exact residuals. That is a plausible product path from stochastic code generation toward governed verified synthesis.

Data and artifact availability

The full per-repository results, cost ledger, issue drafts, artifact hashes, and a private 43-artifact **vero-extract** bundle accompany this draft. The extracted artifacts will be shared privately with the Vero maintainers for independent grading and will not be made public unless requested. Author list, company affiliation, artifact license, source-release boundary, and AI-use disclosure must be confirmed before archival submission.

- Alur, Rajeev, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. “Syntax-Guided Synthesis.” In *Proceedings of FMCAD*.
- Solar-Lezama, Armando, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. “Combinatorial Sketching for Finite Programs.” In *Proceedings of ASPLOS*.
- Ye, Zhe, Hantao Lou, Yuechun Sun, Peiyang Song, Zhengxu Yan, Timothe Kasriel, Qingyang Zhang, et al. 2026. “Vero: Can AI Agents Build Formally Verified Software Repositories?” *arXiv Preprint arXiv:2608.13522*. <https://arxiv.org/abs/2608.13522>.
- Ye, Zhe, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. 2025. “Verina: Benchmarking Verifiable Code Generation.” *arXiv Preprint arXiv:2505.23135*. <https://arxiv.org/abs/2505.23135>.
- Ye, Zhe, Aidan Z. H. Yang, Huangyuan Su, Zhenyu Liao, Samuel Tenka, Zhizhen Qin, Udaya Ghai, Dawn Song, and Soonho Kong. 2026. “Intent-Aligned Formal Specification Synthesis via Traceable Refinement.” *arXiv Preprint arXiv:2604.10392*. <https://arxiv.org/abs/2604.10392>.